

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Recurso) MEIC, MEIM	
Nº	Nome:	20/01/2025

EXAME ÉPOCA RECURSO (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (—). Uma opção assinalada corretamente soma 0,25 val. e incorretamente desconta 0,125 val. (50% da cotação da alínea).

1. [1 val.] Sobre características dos sistemas distribuídos

☐	O teorema CAP afirma que é possível alcançar simultaneamente Consistência, Disponibilidade e Tolerância a Partições
☐	Num sistema <i>stateless</i> , se um servidor falhar, os clientes podem reconectar-se a outro servidor, sem perda de informação
☐	A elasticidade implica sempre escalabilidade, mas a escalabilidade não implica necessariamente elasticidade
☐	Se um sistema tem bom desempenho numa rede local, igualmente terá bom desempenho num ambiente distribuído

2. [1 val.] Considere a operação gRPC: `rpc download(ID) returns (stream Block);` com as mensagens: `message ID { string id = 1; }` `message Block{ bytes data = 1; }`

☐	Na aplicação cliente faz sentido chamar a operação usando um <i>stub</i> não bloqueante, mas também pode ser chamada com um <i>stub</i> bloqueante, retornando um <i>Iterator</i> sobre um <i>stream</i> com blocos de dados
☐	A implementação no servidor retorna um objeto <code>StreamObserver<Block></code> à aplicação cliente para que esta receba dados do servidor como uma sequência de blocos (<i>byte array</i>)
☐	A aplicação cliente chama a operação <code>download</code> com um <i>stub</i> não bloqueante passando um objeto do tipo <i>ID</i> e um objeto do tipo <code>StreamObserver<Block></code> , onde o cliente receberá assincronamente uma sequência de objetos do tipo <i>Block</i> no método <code>onNext(Block bloco)</code>
☐	A implementação no servidor retorna um objeto do tipo <i>Block</i> por cada chamada com um objeto <i>ID</i>

3. [1 val.] Sobre o *middleware* RabbitMQ

☐	Um <i>exchange</i> TOPIC comporta-se exatamente como um <i>exchange</i> DIRECT se não usarmos os caracteres especiais (* e #)
☐	Quando um produtor publica uma mensagem num <i>exchange</i> , recebe como retorno um identificador da <i>queue</i> onde a mensagem será armazenada
☐	Um consumidor pode configurar o consumo de mensagens de um <i>queue</i> em modo <i>auto-acknowledge</i> . No entanto, em caso de erro durante a execução do <i>handler</i> , as mensagens nunca serão repostas na <i>queue</i>
☐	O padrão <i>work-queue</i> pode estar associado a qualquer <i>queue</i> , independentemente do tipo de <i>exchange</i> a que a <i>queue</i> está associada

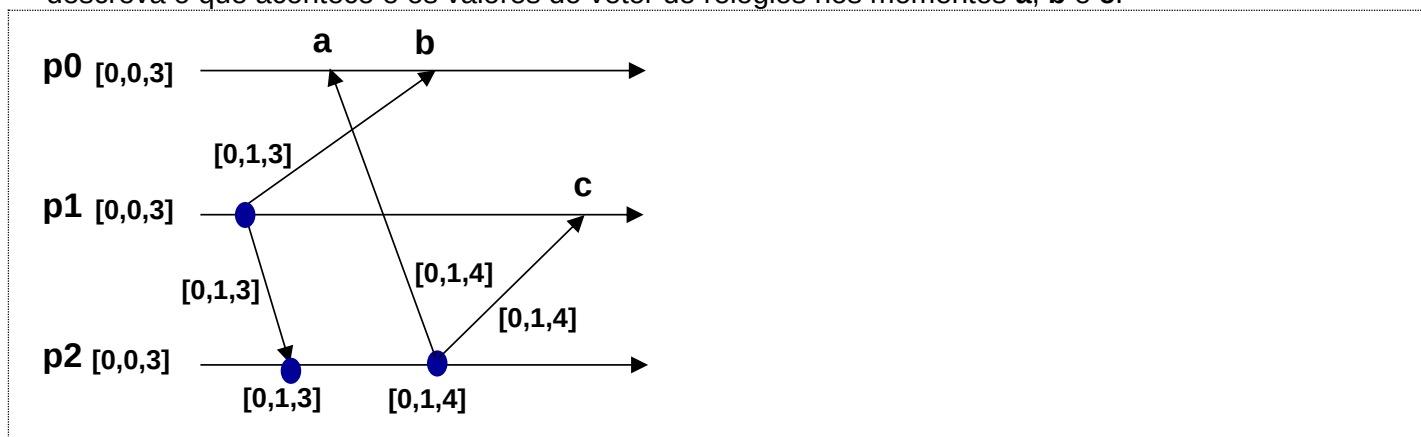
4. [1 val.] Sobre ordenação de eventos, algoritmos de exclusão mútua, eleição e consensos

☐	Para que um algoritmo de eleição seja bem-sucedido é necessário garantir que todos os participantes aceitam qual deles é o líder
☐	Os relógios lógicos de <i>Lamport</i> são suficientes para garantir a ordem real dos eventos em todos os nós
☐	No Protocolo <i>Two-Phase Commit</i> (2PC), o coordenador pode decidir fazer <i>commit</i> por maioria, mesmo que durante a fase de preparação (<i>prepare</i>) um participante vote "não"
☐	O algoritmo de exclusão mútua em anel (<i>Token Ring</i> de <i>Tanenbaum</i>) requer um servidor central para garantir que todos os participantes recebem o <i>Token</i>

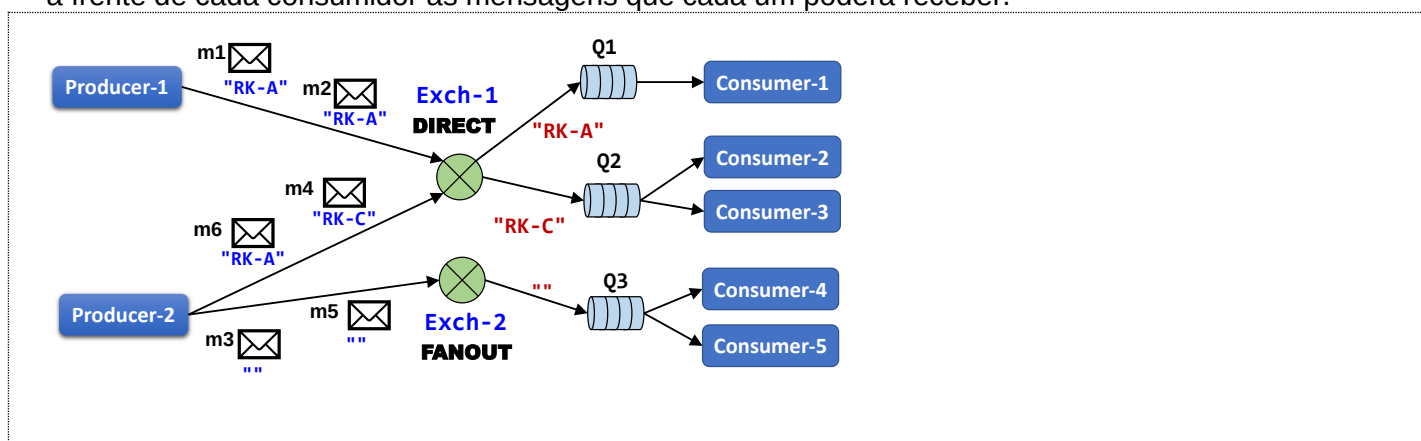
5. [1 val.] Sobre Docker *containers*

☐	O ficheiro <i>Dockerfile</i> serve para executar novos <i>containers</i> através do comando <i>docker run</i>
☐	Um <i>container</i> Docker continua a existir após ser parado, mantendo o seu estado, a menos que seja explicitamente removido
☐	Quando modificamos ficheiros dentro de um <i>container</i> em execução, a imagem original usada para lançar o <i>container</i> é automaticamente atualizada
☐	É possível ter múltiplas versões (<i>tags</i>) da mesma imagem

6. [2 val.] Considere um sistema distribuído, constituído por 3 participantes (**p0**, **p1**, **p2**), que utiliza comunicação por *multicast* fiável e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. Considerando o esquema de troca de mensagens *multicast* apresentado na figura, descreva o que acontece e os valores do vetor de relógios nos momentos **a**, **b** e **c**.



7. [2 val.] Considere um sistema desenvolvido com o *middleware* RabbitMQ que permite aos produtores publicar mensagens nos *exchanges* Exch-1 e Exch-2. Considerando as *routing keys* indicadas nas mensagens **m1**, ..., **m6**, publicadas pelos dois *producers* e as associações (*bindings*) das várias *Queues*, escreva sobre o diagrama da figura o fluxo de encaminhamento dessas mensagens, indicando claramente à frente de cada consumidor as mensagens que cada um poderá receber.



8. [2 val.] Considere a existência de um cluster formado por múltiplas máquinas virtuais (VM) que partilham entre si um volume Gluster, com replicação de ficheiros na diretoria **/var/fileShare**. Assim, qualquer ficheiro escrito ou alterado numa VM será partilhado em todas as VM na diretoria **/var/fileShare**. Em determinado momento foi decidido instalar o Docker em todas as VM, mas surgiu a dúvida se é possível estender a funcionalidade de partilha de ficheiros por todos os *containers* que se executem em todas as VM. Ajude a esclarecer a dúvida indicando uma estratégia para que uma aplicação em execução num *container* possa ler ou escrever ficheiros produzidos por outros *containers*.

